# SOFTWARE DESIGN

## SPRINT 1 RETROSPECTIVE

## 2868072

This sprint was very difficult for me as I came in with limited experience in backend development and had to learn a lot on the job. My three assigned tasks were creating the POST /groups API endpoint (TG-7), building the admin invite form with group assignments (TG-8), and creating the backend API to save pending invites (TG-9). I was also part of the Azure deployment task.

For TG-7 I built the groups route in Express, which handles creating, reading, updating and deleting stokvel groups. This involved setting up the routes/ folder structure, writing the route handlers and updating server.js to register the new routes. For TG-8 I built the admin invite form in the frontend, which allows an admin to enter an email address and assign the user to a group, the send invite functionality is not fully complete. For TG-9 I built the backend API endpoint that receives the invite data, validates it, checks for duplicate emails, and saves the invite with a pending status (it is incomplete as well).

The most difficult part of the sprint for me was understanding how Express works, I had never built a backend before and concepts like routes and how the frontend communicates with the backend through fetch calls were all new to me. I also found Git very challenging, especially when multiple teammates were pushing to the same branch at the same time which caused merge conflicts I had to resolve. The Azure deployment also took a very long time and involved many failed attempts before the app was finally live.

If I could do this sprint again, I would start deployment much earlier rather than leaving it to the last minute. I would also make sure the whole team agrees on the folder structure and commits their files regularly so we avoid the situation where files exist locally but are not on GitHub, which blocked our deployment for a while.

Although all user stories were not fully satisfied because the task distribution among us was not fair, I am proud of the progress I made given that most of the concepts in this sprint were completely new to me, and I am confident that with better team coordination and clearer task ownership in Sprint 2,I hope we will deliver more complete and polished features.

# SOFTWARE DESIGN PROJECT

## SPRINT 2 – RETROSPECTIVE

## 2868072

This sprint I focused on the invite members functionality. I worked on setting up the role dropdown on the invite-members page and integrating the invites.js file with EmailJS to send invitation emails directly to invited users. The integration required setting up an EmailJS account, configuring a service, creating an email template with the correct variables, and connecting it to the frontend using the public key, service ID, and template ID.

The most challenging part of this sprint was dealing with two separate bugs. The first was that groups were not loading in the dropdown, this turned out to be an error in how I was fetching data from the database, which I resolved through research and debugging. The second issue was that the invitation token in the accept link was invalid during local testing, after investigation I found that this was a local testing limitation with how npx serve handles URL parameters, and the token was actually being passed correctly on the deployed Azure site. This taught me the importance of testing on the deployed environment rather than spending too much time debugging on localhost.

One thing I would do differently next sprint is break my tasks into smaller pieces from the start. Some of the issues I ran into took longer than expected because I was trying to solve multiple things at once instead of isolating each problem.

In terms of how my work fits into the overall system, the invite functionality is dependent on the group creation feature my teammate was building. Without groups existing in the database, an admin cannot assign an invited member to any group. In turn, without invites, stokvel groups would not have actual members.

A key thing I learned this sprint was how EmailJS works as a lightweight alternative to supabase edge functions for sending emails. Since this is a non-production academic project, EmailJS gave us a simpler path to working email delivery without the overhead of deploying and managing edge functions, which would be the more appropriate approach for a production system.

# SOFTWARE DESIGN

# SPRINT 3 RETROSPECTIVE

# 2868072 – BOITUMELO NKOSI

This sprint I was responsible for building the contributions functionality, which included implementing contributions.js and contributions.html with PayFast payment integration, as well as improving frontend code coverage across the project. I also contributed to the code coverage setup by writing and improving Jest tests for multiple frontend JavaScript files, helping push our overall coverage above 60%.

The most challenging part of this sprint was integrating PayFast into the contributions page. Understanding how PayFast's sandbox environment works, specifically how to correctly build and auto-submit the hidden form with the right merchant credentials, return URLs, and payment fields , it took a lot of time time and research. Debugging why certain fields were being rejected and ensuring the payment redirect worked correctly in the browser was particularly tricky, but I was able to resolve it by carefully reading through the PayFast documentation and testing each field individually.

If I could do this sprint differently, I would start earlier on the integration tasks rather than leaving them closer to the deadline. I would also communicate more proactively with the team when blocked, rather than spending too much time trying to solve something alone. Planning my tasks at the beginning of the sprint and breaking them into smaller daily goals would have made the workload more manageable.

Despite the challenges, I completed all my assigned Taiga tasks this sprint. The contributions page now supports both member and treasurer views, with members able to pay online via PayFast and treasurers able to confirm or flag contributions. My work fits into the overall system as the financial transaction layer, without working contributions, the core stokvel functionality of collecting and tracking payments cannot operate.

# SOFTWARE DESIGN

# SPRINT 4 RETROSPECTIVE

## Boitumelo Nkosi, 2868072

This sprint I was responsible for two  tasks: building the notifications page and improving the frontend code coverage. For the notifications feature, I implemented the full notifications page that allows users to view all their group activity notifications in one place, including updates about payments, meetings and payouts. Previously notifications were only shown as pop-ups that disappeared after a few seconds, so this page gives users a permanent record of their activity. I built both the notifications.html page and the corresponding JavaScript logic to fetch and display notifications from the database.

The second major task I worked on this sprint was improving frontend Jest code coverage. When I reviewed the existing test files I found that SOME of them were not actually testing the real code, they were reimplementing logic manually inside the test files by copying functions and testing local variables, rather than importing and calling the actual exported functions from the real JS files. This meant the coverage reports were misleading because the real code was never being executed during tests. I rewrote the test files to import and call the actual functions directly, which improved the accuracy and meaningfulness of our coverage. Through this process I also fixed several tests that were failing because they relied on mocked stuff rather than real function execution.

The most difficult part of this sprint was understanding why coverage percentages were low despite having many tests. Debugging the difference between tests that call real functions versus tests that only test inline logic took time and required reading through each test file carefully.

If I could do this sprint differently I would have reviewed the existing test files earlier  so I had more time to rewrite them properly rather than rushing towards the end. I completed all my assigned Taiga tasks this sprint. My work on the notifications page contributes directly to the member experience by ensuring users stay informed about their group activity, and my work on code coverage strengthens the overall quality and reliability of the codebase going into the final submission.